

Algoritmos de Ordenação

Quicksort, Mergesort e Heapsort. Resumo para AEDs II (PUC Minas), baseado nos slides dos Tópicos 1, 2 e 3 e nos códigos da aula.

1. Quicksort

Proposto por Hoare em 1960 e publicado em 1962. É um dos algoritmos de ordenação mais eficientes na prática e segue dividir e conquistar. O vetor é particionado em duas partes que são ordenadas de forma independente, sem etapa final de combinação: a esquerda tem elementos menores ou iguais ao pivô, e a direita, maiores ou iguais.

Analogia: organizar uma fila por altura. Escolhe-se uma pessoa de referência; quem é mais baixo vai para a frente e quem é mais alto vai para trás. Depois, repete-se o processo em cada grupo.

Particionamento (esquema de Hoare)

- i começa em esq e j começa em dir . O pivô é $vet[(esq+dir)/2]$.
- i avança enquanto $vet[i] < pivô$. j recua enquanto $vet[j] > pivô$.
- Se $i \leq j$, troca $vet[i]$ com $vet[j]$ e avança/recua os dois índices.
- Repete-se enquanto $i \leq j$. Ao fim, $quicksort(esq, j)$ e $quicksort(i, dir)$ ordenam cada parte.

```
void quicksort(int esq, int dir) {
    int i = esq, j = dir, pivô = vet[(esq + dir) / 2];
    while (i <= j) {
        while (vet[i] < pivô) i++;
        while (vet[j] > pivô) j--;
        if (i <= j) {
            swap(i, j);
            i++;
            j--;
        }
    }
    if (esq < j) quicksort(esq, j);
    if (i < dir) quicksort(i, dir);
}
```

Exemplo do slide: vetor de 16 elementos

Vetor [0, 1, 5, 3, 15, 16, 9, 10, 4, 3, 30, 5, 20, 48, 71, 82], índices de 0 a 15. Pivô = $vet[7] = 10$.

- i para em 15 (índice 4) e j para em 5 (índice 11). Troca: $vet[4]$ vira 5 e $vet[11]$ vira 15.
- i para em 16 (índice 5) e j para em 3 (índice 9). Troca: $vet[5]$ vira 3 e $vet[9]$ vira 16.
- i para em 10 (índice 7) e j para em 4 (índice 8). Troca: $vet[7]$ vira 4 e $vet[8]$ vira 10.
- Agora $i = 8$ e $j = 7$, então $i > j$ e a partição termina.

Resultado: esquerda [0, 1, 5, 3, 5, 3, 9, 4] (índices 0 a 7), todos menores ou iguais a 10; direita [10, 16, 30, 15, 20, 48, 71, 82] (índices 8 a 15), todos maiores ou iguais a 10. Em seguida, $quicksort(0, 7)$ e $quicksort(8, 15)$.

Exemplo do Quiz 2A: vetor de 7 elementos

Vetor [20, 10, 30, 50, 45, 35, 5]. Pivô = $vet[3] = 50$. A primeira troca é entre 50 e 5, e o resultado da partição é [20, 10, 30, 5, 45, 35, 50].

Escolha do pivô

A aula usa o elemento central. Os slides comparam outras estratégias:

- **Mediana perfeita:** divide o vetor ao meio, mas achá-la exige ordenar, o que anula a vantagem.
- **Média ou moda:** exigem processamento extra sobre o subvetor e, em geral, não compensam.
- **Informação privilegiada:** se a distribuição dos dados é conhecida, técnicas estatísticas ajudam a escolher um pivô melhor.
- **Primeiro ou último elemento:** simples, mas inadequado para vetores ordenados ou inversamente ordenados, pois gera partições totalmente desbalanceadas.
- **Elemento do meio** ($\text{vet}[(\text{esq}+\text{dir})/2]$): evita o pior caso em vetores já ordenados.
- **Pivô aleatório:** qualquer escolha tem probabilidade constante de gerar partição entre um quarto e três quartos do vetor. Por isso, o tempo esperado é $\Theta(n \lg n)$.
- **Mediana de três:** usa o primeiro, o central e o último. O custo extra compensa mais em subvetores grandes.

Complexidade

- **Melhor caso, $\Theta(n \lg n)$.** Cada partição divide o vetor ao meio: $C(n) = 2 \times C(n/2) + n$, com $C(1) = 0$. Pelo Teorema Mestre, a solução é $\Theta(n \lg n)$. Comparações por nível: 8, depois 4+4, depois 2+2+2+2. Cada nível faz cerca de n comparações, e são $\lg n$ níveis.
- **Pior caso, $\Theta(n^2)$.** O pivô é sempre o maior ou o menor do subvetor, então cada partição elimina só um elemento: $n + (n-1) + \dots + 1 = n(n+1)/2$ comparações.
- **Trocas por partição:** no máximo $\text{ceil}(n/2)$, quando os whiles internos são sempre falsos; no mínimo, cerca de 1.
- **Movimentações** (cada troca equivale a 3 movimentações): no melhor caso, $M(n) = 2 \times M(n/2) + c$ dá $O(n)$. O pior caso aparente seria $O(n^2)$, mas não acontece, porque o desbalanceamento máximo e o máximo de trocas não ocorrem juntos. O total real é $O(n \lg n)$, pois cada elemento cai no subproblema menor no máximo $\lg n$ vezes.
- **Memória:** in-place. A pilha de recursão usa $O(\lg n)$ no melhor e no médio caso e $O(n)$ no pior.
- **Estabilidade:** não é estável, porque as trocas podem inverter a ordem de chaves iguais.

2. Mergesort

Algoritmo de divisão e conquista baseado em intercalação. Divide o vetor recursivamente até subvetores de tamanho 1 e depois intercala os subvetores dois a dois. Geralmente é recursivo e usa memória adicional.

Definição: um algoritmo é in-place se requer memória adicional constante, independentemente do tamanho do vetor. Por isso, o Mergesort não é in-place.

Analogia: juntar duas pilhas de cartas já ordenadas. Compara-se o topo das duas pilhas, põe-se na mesa a carta menor e repete-se até acabarem as cartas.

Intercalação: exemplo dos slides

Intercalar [1, 2, 3, 4, 9] com [3, 5, 6, 7, 8]. Compara-se o início de cada lista e o menor vai para o resultado: 1, 2, 3 (da esquerda, por empate), 3 (da direita), 4, 5, 6, 7, 8, 9. Resultado: [1, 2, 3, 3, 4, 5, 6, 7, 8, 9].

Divisão e intercalação: exemplo [8, 3, 6, 1, 5, 4, 2, 7]

- Divisão: [8 3 6 1 | 5 4 2 7], depois [8 3] [6 1] [5 4] [2 7], depois subvetores de 1 elemento.
- Intercalação dos pares: [3 8] [1 6] [4 5] [2 7].

- Próximo nível: [1 3 6 8] e [2 4 5 7].
- Resultado final: [1 2 3 4 5 6 7 8].

Código da aula

```
void mergesort(int esq, int dir) {
    if (esq < dir) {
        int meio = (esq + dir) / 2;
        mergesort(esq, meio);
        mergesort(meio + 1, dir);
        intercalar(esq, meio, dir);
    }
}

void intercalar(int esq, int meio, int dir) {
    int nEsq = (meio + 1) - esq; // tamanho da metade esquerda
    int nDir = dir - meio;      // tamanho da metade direita
    int *vetEsq = (int *) malloc((nEsq + 1) * sizeof(int));
    int *vetDir = (int *) malloc((nDir + 1) * sizeof(int));

    // Sentinela: 0x7FFFFFFF é maior que qualquer elemento
    vetEsq[nEsq] = vetDir[nDir] = 0x7FFFFFFF;

    int iEsq, iDir, i;
    for (iEsq = 0; iEsq < nEsq; iEsq++) vetEsq[iEsq] = vet[esq + iEsq];
    for (iDir = 0; iDir < nDir; iDir++) vetDir[iDir] = vet[(meio + 1) + iDir];

    // Copia sempre o menor dos dois inícios de volta para vet
    for (iEsq = iDir = 0, i = esq; i <= dir; i++)
        vet[i] = (vetEsq[iEsq] <= vetDir[iDir]) ? vetEsq[iEsq++] : vetDir[iDir++];

    free(vetEsq); // liberação (não aparece no código da aula)
    free(vetDir);
}
```

- nEsq e nDir são os tamanhos das duas metades: esq..meio e meio+1..dir.
- Cada metade é copiada para um vetor auxiliar com uma posição extra, que recebe a sentinela 0x7FFFFFFF. Quando uma metade acaba, a sentinela faz a outra ser escolhida até o fim, sem teste extra.
- A comparação usa <=. Em caso de empate, sai primeiro o elemento da esquerda, e por isso o Mergesort é estável.
- O free ao final não aparece no código da aula. Sem ele, a memória alocada não é devolvida.

Na aula, os for copiavam com `vetEsq[iEsq++] = vet[esq+iEsq]`. Isso é comportamento indefinido em C, porque iEsq é alterado e lido na mesma expressão. Aqui está reescrito em duas partes, com o mesmo efeito.

Complexidade

- **Comparações:** cada intercalação faz $\Theta(n)$ comparações, proporcional à soma dos tamanhos dos subvetores. Com $C(1) = 0$ e $C(n) = 2 \times C(n/2) + \Theta(n)$, para $n = 2^k$, o resultado é $\Theta(n \lg n)$.
- **Movimentações:** cada intercalação move os n elementos duas vezes (para os auxiliares e de volta), ou seja, $2n$. Então $M(1) = 0$ e $M(n) = 2 \times M(n/2) + 2n$, o que dá $O(n \lg n)$.
- **Memória:** vetores auxiliares de tamanho $\Theta(n)$ em cada intercalação, e pilha de $\Theta(\lg n)$. Não é in-place.
- **Estabilidade:** sim.

Prova de corretude (resumo dos slides)

- **Lema (intercalação), por indução simples sobre k:** após k iterações, `vet[esq..esq+k-1]` contém, em ordem, os k menores elementos. Base: $k = 0$, nada foi

intercalado. Passo: o próximo menor elemento é $\min(\text{vetEsq}[\text{iEsq}], \text{vetDir}[\text{iDir}])$. Ele entra na posição seguinte, e os $k+1$ menores ficam em ordem.

- **Teorema (Mergesort), por indução forte sobre n:** base $n = 1$, o vetor já está ordenado. Passo: um vetor de tamanho $k+1$ é dividido em partes de tamanhos a e b , com $1 \leq a, b \leq k$. Pela hipótese, as duas partes ficam ordenadas, e pelo lema a intercalação produz um vetor ordenado.
- **Por que indução forte:** as partes têm tamanhos a e b , que não são necessariamente k . Saber só que a propriedade vale para k não basta.

3. Heapsort

Algoritmo de ordenação por seleção. Organiza o vetor como um heap máximo, troca o maior elemento (a raiz) com o último e repete o processo. Buscar o maior a cada passo, como na seleção simples, custa tempo linear. No heap, a raiz já é o maior, e manter a estrutura custa $O(\lg n)$.

Analogia: torneio de eliminação. O campeão (maior elemento) fica no topo. Ao tirar o campeão, o torneio é refeito para escolher o novo campeão.

Vocabulário de árvores

- **Raiz** é o nó do topo, no nível 0. Nós com filhos são internos; nós sem filhos são folhas.
- **Altura h** é o número de arestas entre a raiz e a folha mais distante.
- **Árvore binária:** cada nó tem no máximo dois filhos.
- **Heap:** árvore binária em que cada nó é menor ou igual aos filhos. **Heap máximo:** cada nó é maior ou igual aos filhos, e a raiz é o maior valor. Daqui em diante, 'heap' significa heap máximo.
- **Forma:** o penúltimo nível é completo, e as folhas do último nível estão o mais à esquerda possível.

Heap guardado em vetor

Relação	Base 1 (vet[1] é a raiz)	Base 0 (vet[0] é a raiz)
Raiz	vet[1]	vet[0]
Pai de i	vet[i/2]	vet[(i-1)/2]
Filhos de i	vet[2i] e vet[2i+1]	vet[2i+1] e vet[2i+2]
i é folha se	$i > \text{floor}(n/2)$	$i \geq \text{floor}(n/2)$

Caminho até a raiz: divide-se o índice por 2 até chegar a 1. Exemplo com o vetor [16, 14, 10, 13, 11, 8, 9, 7, 1, 5]: a partir do índice 10, passa-se por vet[10] = 5, vet[5] = 11, vet[2] = 14 e vet[1] = 16. Esse vetor é um heap, porque cada pai é maior ou igual aos filhos.

Inserção

Princípio: cria-se uma nova folha na próxima posição livre do último nível (ou de um novo nível). Se o novo elemento for maior que o pai, troca-se com ele e repete-se com os ancestrais, até o pai ser maior ou igual ao elemento ou até chegar à raiz.

Exemplo do slide: inserir 31 no heap [16, 14, 10, 13, 11, 8, 9, 7, 1, 5]. O 31 entra na posição 11. Troca com o 11 (posição 5), depois com o 14 (posição 2) e depois com o 16 (posição 1). Resultado: [31, 16, 10, 13, 14, 8, 9, 7, 1, 5, 11].

Remoção do maior elemento

- Guarde o valor da raiz em uma variável temporária.

- Coloque na raiz o valor da última folha e remova essa folha.
- Troque a raiz com o maior filho.
- Repita com o elemento trocado até ele ser maior ou igual aos filhos ou chegar a uma folha.

Construção e ordenação

A aula constrói o heap por inserções: `construir(tam)` faz o elemento na posição `tam` subir enquanto for maior que o pai.

```
void heapsort() {
    // Construção do heap: insere os elementos um a um
    for (int tam = 2; tam <= n; tam++)
        construir(tam);

    // Ordenação: a raiz (maior) vai para o fim e o heap diminui
    int tam = n;
    while (tam > 1) {
        swap(1, tam--);
        reconstruir(tam);
    }
}

void construir(int tam) {
    for (int i = tam; i > 1 && vet[i] > vet[i / 2]; i /= 2)
        swap(i, i / 2);    // o elemento sobe enquanto for maior que o pai
}

void reconstruir(int tam) {
    int i = 1;              // começa na raiz
    while (hasFilho(i, tam)) {
        int filho = getMaiorFilho(i, tam);
        if (vet[i] < vet[filho]) {
            swap(i, filho);    // troca com o maior filho
            i = filho;        // o elemento desce
        } else {
            break;             // já é maior ou igual aos filhos
        }
    }
}
```

Funções de apoio dos slides

```
boolean hasFilho(int i, int tam) {
    return i <= tam / 2;    // i tem filho se 2*i <= tam
}

int getMaiorFilho(int i, int tam) { // chamar só quando hasFilho for verdadeiro
    if (2 * i == tam || vet[2 * i] > vet[2 * i + 1])
        return 2 * i;
    return 2 * i + 1;
}
```

Exemplo da aula

Vetor inicial [101, 115, 30, 63, 47, 20]. Depois da construção, o heap fica [115, 101, 30, 63, 47, 20], porque 115 subiu por ser maior que 101. Na primeira extração, troca-se 115 com 20 e reduz-se o heap para 5 elementos. Após `reconstruir(5)`, o vetor fica [101, 63, 30, 20, 47 | 115].

Complexidade

- **Pior caso:** são $(n-1)$ inserções e $(n-1)$ remoções. Cada uma pode percorrer um ramo inteiro, com $\Theta(\lg n)$ comparações. Total: $[(n-1) \cdot \Theta(\lg n)] + [(n-1) \cdot \Theta(\lg n)] = \Theta(n \lg n)$.

- **Melhor caso:** cada inserção faz só uma comparação, mas as remoções continuam custando $\Theta(\lg n)$. Total: $(n-1) \cdot 1 + (n-1) \cdot \Theta(\lg n) = \Theta(n \lg n)$.
- **Movimentações:** $\Theta(n \lg n)$ no pior e no melhor caso, contando cada troca como 3 movimentações.
- **Construção por inserções:** a i -ésima inserção custa $\lg(i)$, então o total é $\lg(1) + \lg(2) + \dots + \lg(n) = \lg(n!)$. Pela aproximação de Stirling, $\lg(n!) \approx n \lg n - \lg(e) \cdot n + (1/2) \lg(2\pi n)$, que é $\Theta(n \lg n)$.
- **Outra forma de construir:** segundo o slide, Cormen et al. (2024) constroem o heap com $\Theta(n)$ comparações. O Heapsort continua $\Theta(n \lg n)$, por causa das $(n-1)$ remoções.
- **Memória auxiliar:** $O(1)$. Não é estável.

4. Comparação

Critério	Quicksort	Mergesort	Heapsort
Paradigma	Dividir e conquistar	Dividir e conquistar	Seleção com heap máximo
Melhor caso	$\Theta(n \lg n)$	$\Theta(n \lg n)$	$\Theta(n \lg n)$
Pior caso	$\Theta(n^2)$	$\Theta(n \lg n)$	$\Theta(n \lg n)$
Memória auxiliar	Pilha: $O(\lg n)$ no melhor caso e $O(n)$ no pior	$\Theta(n)$ de vetores + $\Theta(\lg n)$ de pilha	$O(1)$
In-place	Sim	Não	Sim
Estável	Não	Sim	Não

5. Pontos que costumam cair na prova

- Definição de in-place: memória adicional constante, independente do tamanho do vetor.
- Índices do heap: pai em $i/2$, filhos em $2i$ e $2i+1$, com base 1.
- Partição do Quicksort: i para em elementos $\geq$ pivô, j para em elementos $\leq$ pivô, e troca-se quando $i \leq j$. Pior caso $\Theta(n^2)$ quando o pivô é sempre o extremo.
- Recorrência do Mergesort: $C(n) = 2 \times C(n/2) + \Theta(n)$, que resulta em $\Theta(n \lg n)$. A corretude sai por indução forte.
- Estabilidade: só o Mergesort (com $\leq$) é estável.
- Construção do heap por inserções: $\Theta(n \lg n)$, via $\lg(n!)$. Pela construção de Cormen, $\Theta(n)$.
- Em C, evite escrever $v[i++] = w[i]$ numa mesma expressão. Separe em duas linhas.

Os conteúdos de Quicksort e Mergesort e Heapsort vêm dos slides e dos códigos da aula. A análise de pilha do Quicksort no pior caso e a menção a Cormen et al. (2024) vêm de materiais complementares, então vale confirmar com o professor.